IoT Gateway

Documentation Report Batch 18

https://docs.google.com/document/d/1CXOjGuHlyh_lA--J0WDbbsbSZMItpGcbsGUNG88q2UU/edit?usp=sharing

IoT Gateway is an application for managing IoT sensors, receiving occupancy data from devices via MQTT or HTTP, storing the latest sensor state in SQLite, broadcasting realtime updates with SignalR, and forwarding room occupancy history to the Meeting Room Booking API.

This project consists of:

- `IotGateaway.Api`: ASP.NET Core Web API.
- `IotGateaway.Web`: React/Vite frontend served via Nginx.
- `mosquitto`: MQTT broker for receiving publishes from devices/Raspberry Pi.
- `data`: SQLite database bind mount for the API container.
- `Broker`: Mosquitto configuration, auth, data, and log folders.

Current Features

- Admin login with JWT authentication.
- Sensor/device registration with generated opaque token.
- Sensor listing, detail page, and deletion.
- HTTP ingestion using opaque token authentication.
- MQTT ingestion using Mosquitto and `MqttSubscriberService`.
- Sensor-to-room binding and unbinding.
- Meeting Room Booking room retrieval and device history forwarding.
- Gateway setting management for Meeting Room API credentials.
- SignalR realtime telemetry through `/sensorHub`.
- Single-device live monitor with charts, disconnect, and reconnect controls.
- Sensor inactivity/disconnection notification from MQTT processing.
- Docker Compose setup for API, web, SQLite data mount, and Mosquitto.
- Unit tests for controllers, services, and repositories.

Docker Setup

Run all services:

```
Shell
docker compose up -d --build
```

Run only the API after C# changes:

```
Shell
docker compose up -d --build api
```

Check container status:

```
Shell
docker compose ps
```

View API logs:

```
Shell
docker compose logs -f api
```

View broker logs:

```
Shell
docker logs -f mosquitto
```

Ports used:

Service	Host URL	Container
API	`http://localhost:5000`	`api:8080`
Web	`http://localhost:5173`	`web:80`
MQTT TCP	`localhost:1883`	`mosquitto:1883`
MQTT WebSocket	`ws://localhost:9001`	`mosquitto:9001`

Inside Docker Compose, the API accesses the broker with:

```
```txt
mosquitto:1883
```
```

From the laptop/local host:

```txt
localhost:1883
```

From Raspberry Pi or other devices on the same network, use the IP of the laptop running Docker, for example:

```txt
10.150.1.29:1883
```

Docker Compose

`docker-compose.yml` has 3 main services:

Service	Function
`api`	Builds and runs the ASP.NET Core API. SQLite database is mounted to `/app/data/IotGateway.db`.
`web`	Builds the React app and serves the build output through Nginx.
`mosquitto`	MQTT broker with config, auth, persistence, and logs from the `Broker` folder.

Important API environment values:

```yaml
environment:
    ASPNETCORE_ENVIRONMENT: Development
    ASPNETCORE_URLS: http://+:8080
    ConnectionStrings__DefaultConnection: Data
Source=/app/data/IotGateway.db
    MeetingRoomApi__BaseUrl: https://host.docker.internal:7098/
    MeetingRoomWeb_BaseUrl: https://host.docker.internal:44459/
    Mqtt__Host: mosquitto
    Mqtt__InactivityTimeoutSeconds: 10
```

```
    Cors__AllowedOrigins__0: http://localhost:5173
    Cors__AllowedOrigins__1: http://127.0.0.1:5173
    Cors__AllowedOrigins__2: http://10.150.1.29:5173
    Cors__AllowedOrigins__3: https://localhost:44459
ports:
    - "5000:8080"
volumes:
    - ./data:/app/data
```

Important web build value:

```yaml
args:
    VITE_API_URL: http://localhost:5000/api
```

Important Mosquitto service section:

```yaml
ports:
    - "1883:1883"
    - "9001:9001"
volumes:
    - ./Broker/mosquitto.conf:/mosquitto/config/mosquitto.conf
    - ./Broker/config:/mosquitto/auth
    - ./Broker/data:/mosquitto/data
    - ./Broker/log:/mosquitto/log
```

Broker Setup

Broker config is at `Broker/mosquitto.conf`:

```conf
listener 1883
protocol mqtt

listener 9001
protocol websockets
```

```
allow_anonymous false
password_file /mosquitto/auth/passwd

log_dest stdout
log_dest file /mosquitto/log/mosquitto.log

persistence true
persistence_location /mosquitto/data/
```

This means:

- MQTT TCP listens on port `1883`.
- MQTT over WebSocket listens on port `9001`.
- Anonymous access is disabled.
- Username/password is read from `/mosquitto/auth/passwd`, mounted from `Broker/config/passwd`.
- Persistence data is stored in `Broker/data`.
- Broker logs are stored in `Broker/log/mosquitto.log` and container stdout.

Creating MQTT Users

From Git Bash at the project root:

```shell
MSYS_NO_PATHCONV=1 docker run --rm -it \
  -v "$PWD/Broker/config:/mosquitto/auth" \
  eclipse-mosquitto \
  mosquitto_passwd -c /mosquitto/auth/passwd RaspiProfile1
```

To add another user without overwriting the file:

```shell
MSYS_NO_PATHCONV=1 docker run --rm -it \
  -v "$PWD/Broker/config:/mosquitto/auth" \
  eclipse-mosquitto \
  mosquitto_passwd /mosquitto/auth/passwd AdminIot
```

After modifying the password file:

```shell
docker compose restart mosquitto
```

MQTT Usage

The default API topic is configured in `IotGateaway.Api/appsettings.json`:

```json
"Mqtt": {
  "Host": "mosquitto",
  "Port": 1883,
  "Username": "RaspiProfile1",
  "Password": "Raspi123!",
  "Topic": "sensor/data",
  "InactivityTimeoutSeconds": 30
}
```

When the API runs in Docker, `Mqtt__Host` from Compose points to `mosquitto`. When the API runs locally from Visual Studio or `dotnet run`, `appsettings.Development.json` uses host `localhost`.

Test Subscribe

```shell
docker exec -it mosquitto mosquitto_sub \
  -h localhost \
  -p 1883 \
  -u RaspiProfile1 \
  -P 'Raspi123!' \
  -v \
  -t 'sensor/data'
```

Test Publish From Broker Container

Replace `RAW_OPAQUE_TOKEN` with the raw token returned when creating a sensor.

```shell
docker exec -it mosquitto mosquitto_pub \
  -h localhost \
  -p 1883 \
  -u RaspiProfile1 \
  -P 'Raspi123!' \
  -t 'sensor/data' \
  -m
'{"opaqueToken":"RAW_OPAQUE_TOKEN","hardwareModel":"RaspberryPi-1","isOccupied":tr
ue,"stationaryTargetDistance":120.5,"movingTargetDistance":80.2,"isSensorConnected
":true,"sensorMessage":"Sensor online","updatedAt":"2026-05-19T07:00:00Z"}'
```

Publish Disconnected Sensor Status

If the device wants to explicitly report that the sensor is disconnected, publish with `isSensorConnected: false`:

```shell
docker exec -it mosquitto mosquitto_pub \
  -h localhost \
  -p 1883 \
  -u RaspiProfile1 \
  -P 'Raspi123!' \
  -t 'sensor/data' \
  -m
'{"opaqueToken":"RAW_OPAQUE_TOKEN","hardwareModel":"RaspberryPi-1","isOccupied":fa
lse,"stationaryTargetDistance":0,"movingTargetDistance":0,"isSensorConnected":fals
e,"sensorMessage":"Sensor disconnected","updatedAt":"2026-05-19T07:00:00Z"}'
```

Publish From Raspberry Pi

Replace `10.150.1.29` with the IPv4 of the laptop/PC running Docker:

```shell
mosquitto_pub \
  -h 10.150.1.29 \
  -p 1883 \
  -u RaspiProfile1 \
  -P 'Raspi123!' \
  -t 'sensor/data' \
  -m
'{"opaqueToken":"RAW_OPAQUE_TOKEN","hardwareModel":"RaspberryPi-1","isOccupied":tr
ue,"stationaryTargetDistance":120.5,"movingTargetDistance":80.2,"isSensorConnected
":true,"sensorMessage":"Sensor online","updatedAt":"2026-05-19T07:00:00Z"}'
```

The MQTT payload maps to `MqttSensorRequest`:

```json
{
    "opaqueToken": "RAW_OPAQUE_TOKEN",
    "hardwareModel": "RaspberryPi-1",
    "isOccupied": true,
    "stationaryTargetDistance": 120.5,
    "movingTargetDistance": 80.2,
    "isSensorConnected": true,
    "sensorMessage": "Sensor online",
    "updatedAt": "2026-05-19T07:00:00Z"
}
```

`isSensorConnected` and `sensorMessage` are used by the MQTT subscriber to broadcast connection/status messages to the realtime dashboard. If no MQTT data is received for `Mqtt:InactivityTimeoutSeconds`, the API broadcasts an inactive sensor message through SignalR.

API Entry Point

The main API entry point is at `IotGateaway.Api/Program.cs`.

Things configured in `Program.cs`:

- SQLite DbContext using the `DefaultConnection` connection string.
- ASP.NET Identity for user auth.
- Repository and service registration.
- `OpaqueTokenHelper` and `OpaqueTokenService` for device tokens.
- FluentValidation validators.
- AutoMapper profiles.
- JWT authentication for user/admin HTTP API.
- Opaque token authentication for HTTP ingestion endpoint.
- CORS policy from `Cors:AllowedOrigins`.
- Health endpoint `GET /health`.
- Swagger in Development environment.
- HTTP client named `MeetingRoomApi`.
- SignalR hub at `/sensorHub`.
- Hosted service `MqttSubscriberService` for subscribing to the MQTT broker.

Middleware pipeline:

```txt
ExceptionHandler
Swagger (development)
HTTPS redirection
CORS
Authentication
Authorization
Controllers
SignalR Hub
```

Note: ASP.NET middleware only applies to HTTP requests. MQTT messages do not pass through controllers or HTTP middleware; MQTT is processed by `MqttSubscriberService`.

Response Format

Most services return `ServiceResultDto<T>`:

```C#
public class ServiceResultDto<T>
{
    public T? Data { get; set; }
    public bool Success { get; set; }
    public List<string>? Errors { get; set; }
    public string? Message { get; set; }
}
```

Success response:

```JSON
{
    "data": {},
    "success": true,
    "errors": [],
    "message": "Success"
}
```

Failure response:

```JSON
{
    "data": null,
    "success": false,
    "errors": ["Validation failed"],
    "message": "Validation failed"
}
```

Controllers and Endpoints

Controller	Route	Auth	Function
`AuthController`	`api/auth`	Public	User login and JWT generation.
`SensorController`	`api/sensor`	JWT	Create, retrieve, inspect, and delete sensors/devices.
`IngestionController`	`api/ingestion`	Opaque	HTTP sensor data ingestion and SignalR broadcast. MQTT

Controller	Route	Auth	Function
			ingestion goes through `MqttSubscriberService`.
`ProcessingController`	`api/processing`	JWT	List, bind, and unbind room-sensor mappings.
`MeetingRoomController`	`api/meetingroom`	Public	Retrieve room list from Meeting Room Booking API.
`SettingController`	`api/setting`	JWT	Create/update and retrieve gateway setting/API key for Meeting Room Booking integration.
`UserController`	`api/user`	JWT	Retrieve current authenticated user data.

AuthController

Method	Path	Request	Response
`POST`	`/api/auth/login`	`LoginRequestDto`	`ServiceResultDto<LoginResponseDto>`

`LoginResponseDto` contains the JWT token.

SensorController

Method	Path	Request	Response
`POST`	`/api/sensor`	`DeviceCreateDto`	`ServiceResultDto<DeviceCreateResponseDto>`
`GET`	`/api/sensor`	-	`ServiceResultDto<List<DeviceResponseDto>>`
`GET`	`/api/sensor/{id:guid}`	`Guid id`	`ServiceResultDto<DeviceResponseDto>`
`DELETE`	`/api/sensor/{id:guid}`	`Guid id`	`ServiceResultDto<bool>`

When a sensor is created, the service returns the raw opaque token once and stores only the hashed token in the database.

IngestionController

Method	Path	Request	Response
POST	/api/ingestion	SensorRequestDto	ServiceResultDto<SensorResponseDto>

This HTTP endpoint reads the hashed opaque token from the `opaque_token` claim. For MQTT, this flow is handled by `MqttSubscriberService`.

ProcessingController

Method	Path	Request	Response
GET	/api/processing/bind-room-sensor	-	ServiceResultDto<List<RoomSensorResponseDto>>
POST	/api/processing/bind-room-sensor	RoomSensorBindRequestDto	ServiceResultDto<RoomSensorResponseDto>
DELETE	/api/processing/bind-room-sensor/{id}	Guid id	ServiceResultDto<bool>

MeetingRoomController

Method	Path	Request	Response
GET	/api/meetingroom/rooms	-	ServiceResultDto<List<RoomResponseDto>>

SettingController

Method	Path	Request	Response
GET	/api/setting	-	ServiceResultDto<SettingResponse>
POST	/api/setting	SettingRequest	ServiceResultDto<SettingResponse>

The setting stores the API credential used by `MeetingRoomService` when calling Meeting Room Booking.

UserController

Method	Path	Request	Response
`GET`	`/api/user/me`	-	`ServiceResultDto<UserResponseDto>`

SignalR Realtime Monitoring

SignalR is exposed at:

```txt
http://localhost:5000/sensorHub
```

The hub requires JWT authentication. The web app connects to `/sensorHub`, joins a sensor-specific group using `JoinSensorGroup(sensorId)`, and listens for `ReceiveSensorData`.

Single-device page behavior:

- Opens from a device card.
- Fetches initial sensor detail from `GET /api/sensor/{id}`.
- Connects to SignalR and joins `sensor-{id}`.
- Displays moving target, stationary target, and occupancy charts.
- Shows live status messages from MQTT/ingestion.
- Provides `Disconnect` to stop the SignalR connection.
- Provides `Reconnect` to create a new SignalR connection and join the sensor group again.

Services

Service	Interface	Responsibility
`AuthService`	`IAuthService`	Validate user login and generate JWT.
`SensorService`	`ISensorService`	Create sensor, generate opaque token, retrieve sensor list/detail, and delete sensor.
`IngestionService`	`IIngestionService`	Validate sensor payload, update sensor database state, update

Service	Interface	Responsibility
		room binding, and send history to Meeting Room Booking when needed.
`ProcessingService`	`IProcessingService`	List room-sensor bindings, bind sensor to room, and unbind room sensor.
`MeetingRoomService`	`IMeetingRoomService`	HTTP integration with Meeting Room Booking API, retrieve rooms, and send device history.
`SettingService`	`ISettingService`	Create/update and retrieve gateway setting/API key for Meeting Room Booking integration.
`UserService`	`IUserService`	Retrieve current user data by id.
`DeviceCredentialService`	`IDeviceCredentialService`	Validate hashed opaque token and return device credential.
`OpaqueTokenService`	`IOpaqueTokenService`	Validate raw opaque token from request/payload by hashing the secret token and looking up the device.
`MqttSubscriberService`	Hosted service	Connect to Mosquitto, subscribe to topic, deserialize payload, validate opaque token, ingest data, track inactive sensors, and broadcast.

IngestionService Detail

`IngestionService.IngestSensorDataAsync(string opaqueToken, SensorRequestDto request)` performs:

1. Validate `SensorRequestDto` using FluentValidation.
2. Find sensor by opaque token.
3. Update sensor telemetry fields.
4. Update `UpdatedAt`.
5. Save sensor changes to SQLite.

6. Find `RoomSensor` binding by device id.
7. If no binding exists, return success with a message that sensor update succeeded but no room binding was found.
8. If binding exists, update room sensor occupancy and device model.
9. Send history to Meeting Room Booking if occupancy changed or the 1-minute send interval has passed.
10. Return `ServiceResultDto<SensorResponseDto>`.

MqttSubscriberService Detail

`MqttSubscriberService` runs automatically when the API starts because it is registered with:

```
C#
builder.Services.AddHostedService<MqttSubscriberService>();
```

MQTT flow:

```txt
Raspberry Pi / publisher
-> Mosquitto broker
-> MqttSubscriberService
-> Deserialize MqttSensorRequest
-> OpaqueTokenService.HandleAuthenticateAsync
-> IngestionService.IngestSensorDataAsync
-> SQLite + MeetingRoom API
-> SignalR ReceiveSensorData broadcast
```

The hosted service tracks the latest payload time for each sensor in memory. If a sensor does not publish within `Mqtt:InactivityTimeoutSeconds`, it broadcasts an inactive message through SignalR.

Because the hosted service is a singleton, scoped services like `IIngestionService` and `IOpaqueTokenService` are resolved via `IServiceScopeFactory` when a message is received.

Main DTOs

MqttSensorRequest

Used for MQTT payloads:

```C#
public record MqttSensorRequest
{
    public string? HardwareModel { get; }
    public string? OpaqueToken { get; }
    public bool? IsOccupied { get; }
    public float? MovingTargetDistance { get; }
    public float? StationaryTargetDistance { get; }
    public bool? IsSensorConnected { get; }
    public string? SensorMessage { get; }
    public DateTimeOffset? UpdatedAt { get; }
}
```

SensorRequestDto

Used for HTTP ingestion and as the internal mapped DTO from MQTT:

```C#
public record SensorRequestDto
{
    public string? OpaqueToken { get; }
    public string? HardwareModel { get; }
    public bool IsOccupied { get; }
    public float StationaryTargetDistance { get; }
    public float MovingTargetDistance { get; }
    public DateTimeOffset UpdatedAt { get; }
}
```

`UpdatedAt` must be UTC and must not be in the future.

DeviceCreateDto

Used to create a sensor:

```JSON
{
    "hardwareModel": "LD2410 Sensor"
}
```

RoomSensorBindRequestDto

Used to bind a room and sensor:

```JSON
{
```

```
    "roomId": "GUID_ROOM",
    "sensorId": "GUID_SENSOR"
}
```

SettingRequest

Used to store Meeting Room Booking API credentials:

```JSON
{
    "name": "Meeting Room API",
    "key": "API_KEY_VALUE"
}
```

Database

The API uses SQLite. When running in Docker, the database is located at:

```txt
data/IotGateway.db
```

Container connection string:

```txt
Data Source=/app/data/IotGateway.db
```

Main tables:

- `Sensors`
- `RoomSensors`
- `Settings`
- ASP.NET Identity tables such as `AspNetUsers`, `AspNetRoles`, and related tables.

Health Check

Endpoint:

```shell
curl http://localhost:5000/health
```

Healthy response:

```json
"Service is Running"
```

Common Issues

API Not Receiving MQTT Payload

Check the following:

- API container is `Up`.
- API has been rebuilt after C# changes:

```shell
docker compose up -d --build api
```

- Publisher topic matches `Mqtt:Topic`.
- If `Mqtt:Topic` is `sensor/data`, publish must target `sensor/data`.
- Opaque token in payload is the raw token returned when the sensor was created.
- MQTT username/password exists in `Broker/config/passwd`.

Realtime Page Not Updating

Check the following:

- User is logged in, because `/sensorHub` requires JWT.
- Web app `VITE_API_URL` points to the running API.
- CORS allowed origins include the web URL.
- The page was opened from the device card so the sensor ID exists in the route.
- The device is publishing to the subscribed MQTT topic.
- Use the `Reconnect` button on the single-device page after a manual disconnect or network interruption.

Sensor Shows Inactive

The API broadcasts an inactive message when no MQTT data is received for `Mqtt:InactivityTimeoutSeconds`. In Docker Compose this is currently overridden to `10` seconds.

Possible causes:

- Device stopped publishing.
- Broker is not reachable.
- Topic does not match.
- Device token is invalid.
- MQTT client lost connection.

Broker Logs `First packet not CONNECT`

This usually happens because port `1883` is accessed via HTTP/browser/curl. Port `1883` is for MQTT TCP only. For WebSocket use `ws://localhost:9001`.

`No such host is known: mosquitto`

The host `mosquitto` is only valid from inside Docker Compose containers. If the API runs from Visual Studio or locally, use `localhost`.

DB Not Updated After MQTT Publish

Check API logs:

```shell
docker compose logs -f api
```

Possible causes:

- Opaque token in payload is invalid.
- Payload failed to deserialize into `MqttSensorRequest`.
- Validation failed.
- Sensor is not registered in the database.
- Topic does not match.
- API image has not been rebuilt.

Development Commands

Build solution:

```shell
dotnet build IotGateaway.sln
```

Run API locally:

```shell
dotnet run --project IotGateaway.Api
```

Run web locally:

```shell
cd IotGateaway.Web
npm install
npm run dev
```

Build web:

```shell
cd IotGateaway.Web
npm run build
```

Build Docker API:

```shell
docker compose up -d --build api
```

Restart broker:

```shell
docker compose restart mosquitto
```